



Figure 1: Chef architecture

```
[root@192.168.122.55 ~]# knife bootstrap 192.168.122.55 -v root -P root123 -B 'demo-node'
Being old-style registration with the validation key at chef/chef-repo/chef/chef-validator.pem...
Delete the validation key in order to use your user credentials instead

Connecting to 192.168.122.55
192.168.122.55 -> Booting Chef installation detected
192.168.122.55 Starting the First Chef Client run...
192.168.122.55 Starting Chef Client version 12.8.6
192.168.122.55 Creating a new client identity for demo-node using the validator key.
192.168.122.55 Retrieving cookbooks for run list: []
192.168.122.55 Synchronizing Cookbooks:
192.168.122.55 -> Installing Cookbooks...
192.168.122.55 [2015-12-29T08:16:18-05:00] WARN: Node demo-node has an empty run list.
192.168.122.55 Converging 0 resources
192.168.122.55
192.168.122.55 Running handlers:
192.168.122.55 -> Running handlers complete
192.168.122.55 Chef Client finished, 0/0 resources updated in 00 seconds
[root@192.168.122.55 ~]#
```

Figure 2: Screen shot for Bootstrap execution

'cookbook_101'. It contains code examples for different core resources, which are built into the Chef-clients. A resource can be of different types like a file, directory, command, network, script, package, service, etc, and these resources are grouped together into recipes, which will decide what configuration we want to set up for the target nodes. You should have domain expertise to understand your project's requirements, and should know definitely what state your system should be in to solve your business problems. Please note that to understand the given code examples in this article, the reader should have a fair knowledge of UNIX commands and script writing.

Bootstrap the Chef-client node

First, we need to set up Chef in the target node by using the Bootstrap command as shown in Figure 2. Knife is the command to execute the Bootstrap command in the client system 192.168.122.55 with user as the root, the password root123 and name of the node set as 'demo-node'. If the Bootstrap command is executed successfully, you will see the output shown in Figure 2. From the figure, we can see that the *run list* is empty and no resources have been updated in the client system.

Now let's assume a systems administrator wants to perform the following tasks: configure *n* nodes to create a file, write some content into a file, create a directory to a given path, create a symbolic link for a file to find the total number of directories to a given path and store the value into a file, execute a given shell script or group of commands, copy a

```
# 1. To create a Regular file
file '/home/raju/regular_file' do
  owner 'raju'
  content 'Welcome to the world of Chef'
end
```

Figure 3.1: Code for file creation

```
# 2. To create a directory
directory '/home/raju/dir_file' do
  owner 'root'
  group 'root'
  mode '0744'
  action :create
end
```

Figure 3.2: Code for directory creation

```
# 3. To create a Soft link file
link '/home/raju/symlnk_list.txt' do
  to '/home/raju/list.txt'
  link_type :symbolic
end
```

Figure 3.3: Code for symbolic link file creation

file content into some file, take backups of certain files, stop or start some service, set up a static IP address for a network device, create a new user and create a cron job to automate certain tasks into the nodes. Let us see how to configure the above resources in the form of a recipe in the cookbook_101.

Cookbook_101: Example code

In Figure 3.1, a file is a resource, which will create a 'regular_file' in */home/raju* directory, the owner of which is Raju, and write some content into the file. The code given in this figure is part of cookbook_101 to help create a file with certain properties. The block of code between *do* and *end* typically executes as a single command.

The example code given in Figure 3.2 depicts how to create a directory, 'dir_file', with the directory resource, which has the properties of the owner, group and mode of the directory. The action is to create a directory if it has not already been created. The action may be deleted or 'nothing'. The 'nothing' attribute in an 'action' specifies 'don't do anything unless some other resources invoke some action.' If action is not included in the block of code, then Chef will execute the default action based on the resource type, which may create a file or directory, install a package or start the service.

The example code block given in Figure 3.3 will create a symbolic link file. Link is the resource type; by default, it will create a symbolic link. If you want to create a hard link file, you need to explicitly specify *link_type* as *hard*. The source file is *list.txt* and we want to create a symlink file as *symlnk_list.txt*. If the *link_type* is symbolic, then including *link_type* is optional. Typically, if properties like user, mode, action, etc, are not mentioned in the resource block, they will be assigned their default value.